# VEDHA AI - PROJECT PORTFOLIO & ARCHITECTURE REPORT

A Private, Secure, 100% Offline Retrieval-Augmented Generation (RAG) Learning Assistant

Doc Type: Project Report
Date: August 2026
Confidentiality: Internal
Deployment: Local CPU

### PROJECT OVERVIEW & CORE VALUES

Vedha AI addresses data privacy risks and internet dependencies of standard cloud-based AI systems. Operating entirely offline, it allows users to securely upload, scan (OCR), search, and chat with academic text, PDFs, and slide decks. It guarantees that sensitive corporate, personal, or academic documents are never sent to external servers, providing an offline learning assistant that works anywhere without an internet connection.

### KEY SYSTEM CAPABILITIES & SPECS

- Local Ingestion: Drag-and-drop file ingestion support for PDF, DOCX, PPTX, and Image formats.
- Offline OCR: Baidu PaddleOCR transcribes scanned pages and figures locally on CPU.
- SSE Chat Streaming: streams words in real-time with reference citation context drawers.
- Telemetry Dashboard: displays real-time SQLite size, Chroma chunks, and database analytics.

## TECHNOLOGY STACK & ARCHITECTURE MATRIX

### FRONTEND CLIENT

React 18 + TS
Vite Dev Tooling
Tailwind CSS
Framer Motion
Lucide Icons
Firebase Session

### BACKEND ENGINE

FastAPI (Python)
Uvicorn Server
SQLAlchemy ORM
SQLite Database
aiofiles Async IO
app.log Streamer

### VECTOR & DB

Chroma Vector DB
SentenceTransformers
all-MiniLM-L6-v2
384-Dim Embeddings
Persistent Storage
Chunks Registry

### LOCAL AI CORE

Ollama LLM Engine
qwen2.5:3b model
PaddleOCR (OCR)
PyMuPDF Parsers
Kokoro-82M TTS
SSE Streaming

## CORE ARCHITECTURAL PROCEDURAL FLOWS

### 1. INTENTIONAL DOCUMENT INGESTION PIPELINE

1. Pre-Analysis: FastAPI server extracts initial raw characters from uploaded documents.
2. Topic Classification: Local Ollama processes characters, generating suggested category labels (e.g. Physics).
3. User Confirmation: User reviews the generated metadata and selects/confirms target collection folder.
4. Chunking & Overlap: Recursive splitter splits documents into 512-token segments (64 overlap).
5. Vectorization: sentence-transformers generates embedding vectors stored locally inside ChromaDB.

### 2. OFFLINE RETRIEVAL-AUGMENTED GENERATION

1. Query Ingestion: User enters a prompt in the React chat module, passing the selected collection ID.
2. Similarity Search: System queries local ChromaDB for the 4 most relevant text blocks.
3. Context Injection: Retracted context is integrated into a system prompt. Non-matching queries get a strict refusal prompt, eliminating hallucinations.
4. Model Inference: Context-injected prompt feeds into local Ollama runner to generate answers offline.
5. Real-Time Streaming: SSE yields tokens word-by-word with database citations to the UI.

## PROJECT ACHIEVEMENTS & KEY IMPLEMENTATIONS

- 100% Offline Integrity & Setup Helpers: Replaced remote cloud dependencies with local equivalents (PaddleOCR for images, sentence-transformers for vector embeddings). The settings view tests connectivity to Ollama and displays setup helper CLI commands with a clipboard copy utility when services are offline.

- Dynamic Telemetry Loop: The analytics dashboard polls database sizing, active models, document format ratios, and total Chroma vector counts directly from SQLite and ChromaDB, updating an interactive frontend SVG ring every 4 seconds.

- Database Chunks Registry: Built a dedicated database chunks inspector view (/chunks) that retrieves raw vector database records, enabling transparency. Users can search and filter text chunks with highlighted keyword matches.

- Optimized Scrolling Layouts: Refactored the outer UI layout shell by removing strict h-screen constraints. This resolves dashboard clipping, allowing smooth scrolling across data-intensive telemetry and document registries.